Chuỗi khối và Sổ cái phân tán - Thực hành 2

Lưu ý:

- Sử dụng một trong những ngôn ngữ lập trình sau: Python, C/C++, C#, Java.

- Nộp bài vào cuối buổi học vào email: namtx@wru.vn, tiêu đề là "Bài thực hành Blockchain 2", nội dung bên trong gồm họ tên sinh viên, mã sinh viên, lớp; lời giải viết trực tiếp trong email (phân tách riêng từng bài), nếu lời giải để ở dạng file gắn trực tiếp vào email có thể bị hệ thống chặn, sinh viên sẽ không được tính điểm.

- Trường hợp không làm được bài tập nào, không cần gửi email, nhưng nếu nộp bài thì bài tập phải do chính bạn làm, *không sao chép của người khác, không sử dụng AI giải hộ, sinh viên sẽ bị 0 điểm phần bài tập quá trình nếu vi phạm nguyên tắc này*.

Bài 1. Chuỗi block đơn giản (chỉ có previous_hash)

Viết chương trình mô phỏng một chuỗi block đơn giản như mô tả dưới đây.

Mỗi block chứa:

- index (số thứ tự)
- data (chuỗi mô tả, ví dụ Alice;5;Bob với ý nghĩa "Alice chuyển cho Bob 5 đồng", lưu ý giá trị chuyển có thể là số thực, sử dụng dấu chấm để phân cách phần thập phân)
- prev_hash (hash của block trước)
- hash (tính từ index + data + prev_hash)
- Block đầu tiên (genesis) có prev_hash = "0".

Viết các hàm:

- Hàm add(…): Thêm block mới vào chain (tự động tính prev_hash & hash).
- Hàm isOK(): Kiểm tra tính hợp lệ của toàn bộ chain (hash đúng + prev_hash của block i phải bằng hash của block i-1).
- Hàm trace(): Trường hợp chuỗi hợp lệ thì không in ra gì cả, trường hợp chuỗi in ra toàn bộ chuỗi block (mỗi block trên 1 dòng) và kết quả kiểm tra của từng dòng.

Viết chương trình chính minh họa cách sử dụng các hàm trên.

Bài 2. Tính số dư dựa trên mô hình UTXO đơn giản

Một giao dịch chuyển tiền gồm có thông tin như sau:

- "from": gửi từ ví nào
- "to": nhận vào ví nào
- "amount": số tiền bao nhiêu

Danh sách các giao dịch sẽ có dạng:

from = "COINBASE", to = "Alice", amount = 10

from = "Alice", to = "Bob", amount = 4

from = "Alice", to = "Carol", amount = 3

from = "Bob", to = "Dave", amount = 2

…

Viết chương trình đọc danh sách giao dịch như trên từ file “trans.txt”. Sau đó in ra số dư cuối cùng của mỗi ví. Giả thiết ban đầu mỗi ví đều có 0 coin.

Bài 3. Sắp xếp giao dịch theo phí

Thông tin của mỗi giao dịch gồm:

- id (chuỗi)
- cỡ (kB)
- phí (satoshi)

Hệ thống xử lý giao dịch sẽ ưu tiên chọn giao dịch có rate cao nhất từ công thức: rate = phí / cỡ.

Ví dụ:

Giao dịch A: cỡ = 250 kB, phí = 2500 ⇒ rate = 10

Giao dịch B: cỡ = 100 kB, phí = 1500 ⇒ rate = 15

→ B nên được ưu tiên hơn A.

Viết chương trình:

- Nhập một danh sách giao dịch từ file, mỗi dòng là thông tin một giao dịch.
- Sắp xếp các giao dịch giảm dần theo rate.
- Chọn từ đầu đến cuối danh sách càng nhiều giao dịch càng tốt nhưng tổng cỡ của tất cả các giao dịch không quá 1 MB (1000 kB).

In ra danh sách giao dịch được chọn, tổng cỡ, tổng phí thu được.

Bài 4. Proof-of-Work đơn giản bằng leading zeros

Cho trước một data (ví dụ "Block #1: Alice pays Bob 5").

Tìm một số nguyên nonce sao cho: hash(data + nonce) (SHA-256) có ít nhất N số 0 ở đầu trong dạng hex.

N được nhập từ bàn phím (ví dụ N = 3, 4, 5).

In ra:

- Nonce tìm được.
- Hash tương ứng.
- Số lần thử (số vòng lặp).

Ví dụ:

Input: data = "Hello PoW", N = 3

Output:

nonce = 58291

hash = 000f3a9c...

trials = 58292

Bài 5. Tạo địa chỉ ví (Address Generation)

(Mô phỏng quy trình tạo Bitcoin Address từ Public Key)

Viết chương trình chuyển đổi một Public Key (giả lập) thành một địa chỉ ví chuẩn (dạng Base58Check).

Input: Một chuỗi đại diện cho Public Key (ví dụ: một chuỗi Hex ngẫu nhiên).

Quy trình xử lý như sau (đã đơn giản hóa):

1. SHA-256: Hash Public Key.

2. RIPEMD-160: Hash kết quả bước 1 (nếu không có thư viện tính RIPEMD160, có thể dùng MD5 hoặc cắt chuỗi SHA256 còn 20 bytes để mô phỏng).

3. Thêm Version Byte: Thêm byte `0x00` (đại diện cho Mainnet) vào trước kết quả bước 2.

4. Checksum:

- Hash (SHA-256) kết quả bước 3 hai lần liên tiếp.

- Lấy 4 byte đầu tiên của kết quả hash đó làm checksum.

5. Nối chuỗi: Kết quả bước 3 + Checksum.

6. Base58 Encode: Mã hóa kết quả bước 5 bằng thuật toán Base58.

Output: Chuỗi địa chỉ ví hoàn chỉnh.